

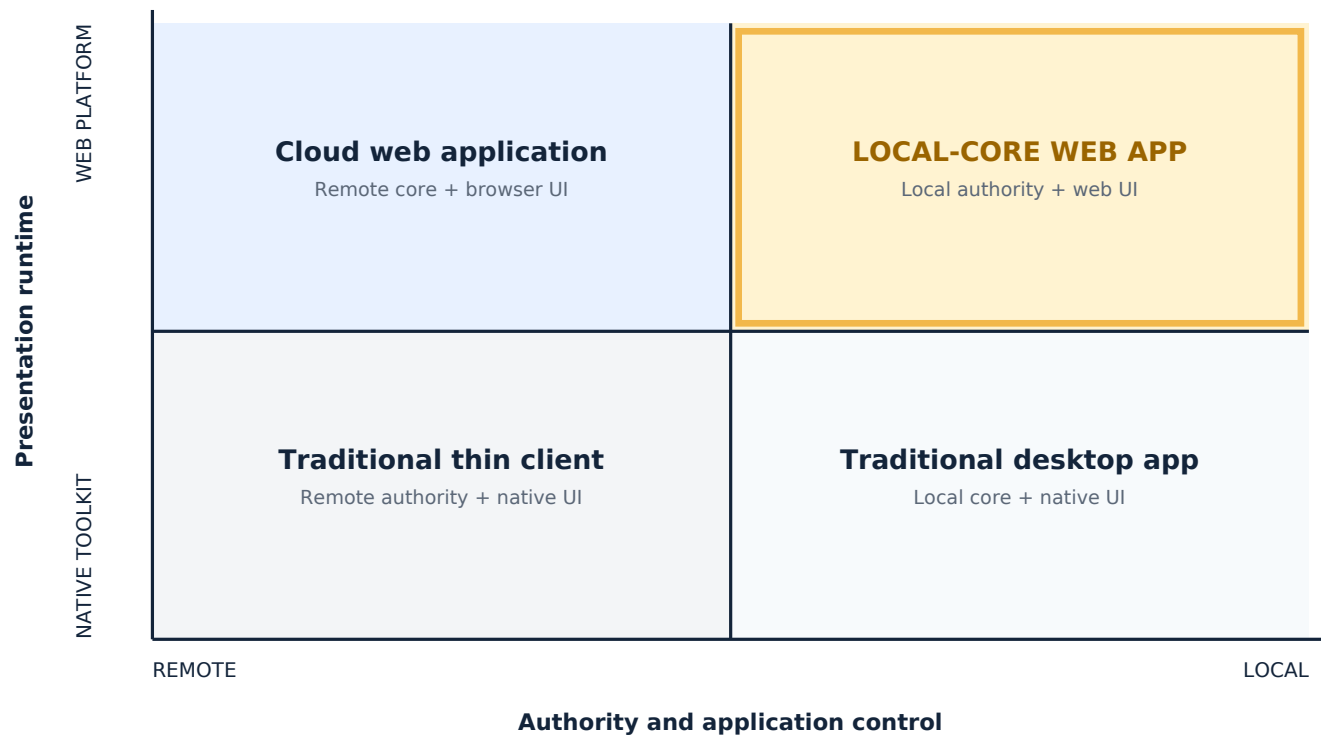
THE LOCAL-CORE WEB APPLICATION

Recognizing the Web Platform as a general-purpose UI runtime - and restoring local authority to personal software.

A paradigm proposal and architectural rationale

CENTRAL THESIS

Web technology is a presentation architecture, not a deployment architecture.



Executive summary

Software is commonly classified along a single line: native application versus web application. That classification hides two separate architectural decisions: where application authority resides, and what technology renders the interface. Once those decisions are separated, an important and under-described category becomes visible: an installed, locally authoritative application whose interface is rendered by the Web Platform.

This paper names that category the **local-core web application**. It joins two independent ideas:

- **Web-platform UI architecture:** a browser engine is treated as a general-purpose presentation runtime. HTML, CSS, JavaScript and the DOM replace the operating-system GUI toolkit.
- **User-sovereign local-first architecture:** the application's essential logic and authoritative data reside on the user's machine. Remote services may enhance the product, but they do not grant it permission to exist.

Either idea can be used without the other. Their intersection, however, combines the portability and expressive power of modern web interfaces with the ownership, durability and independence of traditional personal software.

THE ELEVATOR PITCH

Build desktop software with the UI framework developers already know best: the Web Platform. Keep the application's brains and data on the user's machine, so it remains fast, durable and useful even if its maker disappears. The result has the interface economics of the web and the ownership model of personal computing.

The overlooked distinction

An SPA, a multi-page application and a server-rendered site describe presentation and navigation behavior. They do not determine whether the authoritative application is local or remote. Likewise, a WebView describes a rendering mechanism, not an ownership model. The local-core web application is therefore not a subtype of SPA or a compromise between native and web. It appears only after the classification itself is corrected.

1. Two independent ideas

The paradigm must not conflate its presentation architecture with its authority model. Keeping them separate makes the concept applicable beyond any framework, language or delivery mechanism.

The Web Platform as UI runtime

In conventional web thinking, the browser is the client of an application that lives elsewhere. In web-platform UI architecture, the browser engine has a different role: **it is the application's UI runtime**. The application may use a full browser window, an embedded WebView or another standards-based renderer.

The framework is not merely JavaScript. It is the integrated system of HTML semantics, CSS layout, DOM events, accessibility, typography, media, graphics, input handling, networking primitives and developer tooling supplied by a mature browser engine.

Traditional GUI stack	Web-platform UI stack
Win32, Cocoa, GTK, Qt or another widget toolkit	HTML, CSS, DOM and browser rendering engine
Toolkit-specific widgets and layout	Standards-based document and layout model
Platform bindings and event loop	JavaScript events, APIs and an application bridge
OS-dependent rendering behavior	Cross-platform rendering behavior with a shared skill base

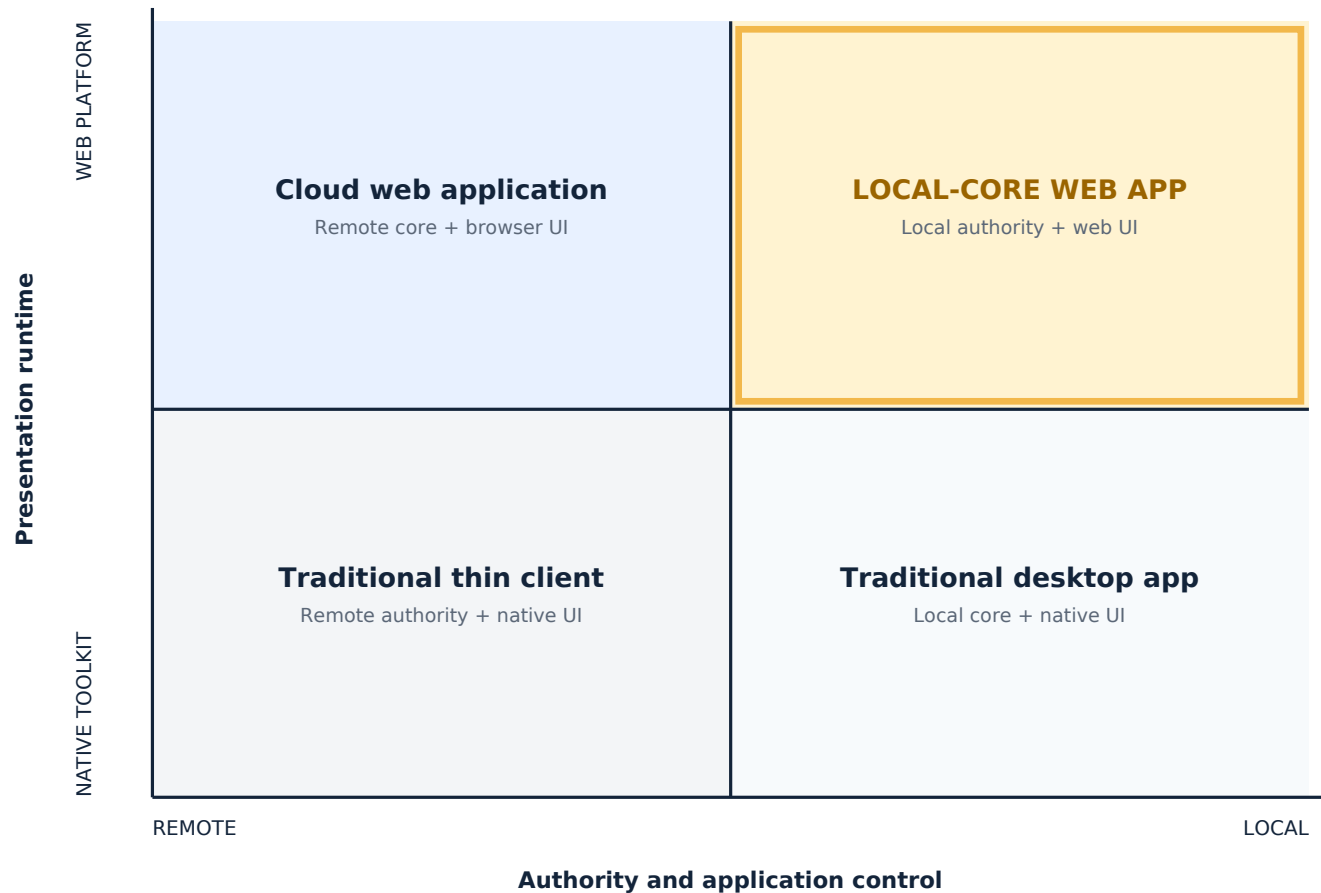
User-sovereign local-first software

Local-first is commonly associated with offline operation or synchronized local databases. The stronger claim advanced here is durability and control: the user possesses a complete working instance of the application, not merely a cached client of a vendor-operated service.

- The essential business logic executes locally.
- The authoritative data is stored under the user's control.
- The installed version remains usable if its maker closes, changes direction or withdraws service.
- Remote infrastructure supplies optional capabilities rather than existential permission.
- Open source can make the execution mechanism inspectable and alterable, rather than permanently vendor-controlled.

2. The missing quadrant

The familiar native-versus-web spectrum combines authority and presentation into one misleading axis. Separating them yields four categories.



The upper-right quadrant is the overlooked category: **local authority plus web-platform presentation**. The interface may use SPA techniques, server-rendered fragments or document navigation. Those are internal presentation choices, not the application's architectural identity.

Why it has remained difficult to see

- **Web app** conventionally implies remote delivery and remote authority.
- **Desktop app** conventionally implies native widgets or a platform GUI toolkit.
- **Hybrid app** describes mixed implementation technology but says nothing about who controls the application or its data.
- **Local-first** usually emphasizes data placement, not the browser engine's role as a GUI framework.
- **Electron app, Tauri app** and similar labels name implementations rather than an architectural category.

The category becomes obvious when we accept that web technology does not logically require remote deployment. A browser engine can render a wholly local application just as Qt or Cocoa can.

3. Anatomy of a local-core web application

A local-core web application has a durable application core and a web-rendered presentation layer. The boundary between them may be HTTP, WebSocket, message passing, direct IPC or a framework-provided command bridge.

WEB-PLATFORM PRESENTATION
Views · interaction · layout · accessibility · transient display state
LOCAL APPLICATION BOUNDARY
localhost HTTP/WebSocket · IPC · command bridge
LOCAL AUTHORITATIVE CORE
business rules · durable state · files · networking · background work · security
OPTIONAL NETWORK
peers · mail servers · relays · synchronization · discovery · updates

A local HTTP server does not make this a conventional client/server product

Mechanically, a WebView requesting resources from localhost resembles client/server communication. Architecturally, both components belong to one installed application, run on one machine, share one lifecycle and operate within one local trust boundary. The 'server' is an internal adapter between the presentation runtime and the application core.

The network changes role

In a cloud application, the network reaches the application. In a local-core application, the application reaches the network. Remote systems may carry messages, synchronize devices or provide public data, but the application itself remains present when those systems are unavailable.

An email application illustrates the distinction

A local-core mail application can maintain its database, search indexes, rules, drafts, contacts and interface locally. SMTP, IMAP or peer protocols become external communications services. The UI does not need to know whether an operation touches a local database or a remote mailbox; it issues commands to the local core.

4. Why this paradigm deserves to exist

The proposal is not that every application should be local-core or that a WebView is always superior. The claim is that this architecture deserves recognition as a first-class default for personal, creative, communication and productivity software.

A modern default UI framework

The Web Platform is already the most broadly learned interface environment in software development. It offers cross-platform rendering, responsive layout, mature accessibility primitives, sophisticated typography and graphics, and an unusually deep ecosystem. Treating it as a GUI framework allows applications to reuse that collective investment without making themselves cloud services.

Durability as a product property

Traditional installed software gave each released version permanence. A user could keep Version 3.2 after its developer shipped Version 4, changed its business model or disappeared. Cloud software weakened that property because the vendor retained the only complete operational instance. A local-core application restores software possession: the user's installed release continues to embody the rules under which it was acquired.

Ownership rather than tenancy

When the authoritative data and execution mechanism live locally, the user's relationship to the product changes. The vendor writes and distributes software; it does not continuously mediate permission to use the user's own work. Export and interoperability remain important, but they are no longer emergency escape hatches from a remote authority.

Operational and economic independence

A product whose essential work happens on users' machines can reduce permanent infrastructure obligations. Remote services can be scoped to the capabilities that actually require coordination. This creates room for small teams, long-lived tools and products whose continued usefulness is not tied to indefinite cloud revenue.

A cleaner architecture

The model gives the presentation layer one job: translate application state into a human interface and user intent into commands. The local core owns durable state and rules. This separation permits the UI to evolve using web techniques without forcing business authority into browser code or a distant server.

5. Boundaries, objections and design test

Recognizing a paradigm requires stating what it is not.

It is not simply an Electron application

Electron, Tauri, WebView2, WKWebView and comparable technologies can implement web-platform UI architecture. None guarantees local authority, durable operation or user ownership. Framework choice is subordinate to the paradigm.

It is not necessarily an SPA

The presentation may be an SPA, a multi-page interface, server-rendered HTML or a mixture. SPA describes how UI transitions are produced; local-core describes where the application lives and who controls it.

It is not isolation from the Internet

A locally authoritative application may be intensely networked. IRC clients, early instant messengers and Thunderbird are useful ancestors because their rich interfaces and substantial behavior were installed locally. Thunderbird is especially close: local databases, indexes and rules are combined with remote communication protocols. The proposed progression replaces the OS-dependent GUI toolkit with the Web Platform.

The local-core test

An application belongs in the local-core category when the following counterfactual is substantially true:

If the maker permanently disappears tomorrow, the installed application can continue performing its essential local purpose with the user's existing data.

A practical checklist

- Can the user launch the installed application without contacting its maker?
- Does the local core hold the authoritative copy of essential user-created data?
- Can the essential business rules execute without a vendor-controlled backend?
- Are network services separable capabilities rather than the application's place of existence?
- Can the UI runtime be replaced or upgraded without relocating authority?
- Can users preserve, export and, where licensing permits, inspect or modify the execution mechanism?

6. A category worth naming

The local-core web application is not a nostalgic return to pre-web desktop development. It is a continuation of personal computing using the most capable cross-platform presentation framework now available.

Its first principle is architectural clarity:

Web technology is a presentation architecture, not a deployment architecture.

Its second principle is user sovereignty:

The user should possess a working application and its authoritative data, not merely permission to visit a service.

Together these principles describe software that is visually modern, portable and connected while remaining locally real. The browser engine becomes the successor to the operating-system GUI toolkit. The local core becomes the durable home of logic and data. The Internet becomes something the application uses rather than the place where the application is allowed to exist.

That combination deserves to be recognized, discussed and developed as a distinct paradigm.

Working terminology

Web-platform UI architecture	Use of a browser engine as a general-purpose GUI runtime.
User-sovereign software	Software whose essential capability and authoritative data remain under the user's control.
Local-core web application	Their intersection: a locally authoritative application presented through the Web Platform.

Selected references

Kleppmann, M.; Wiggins, A.; van Hardenberg, P.; and McGranaghan, M. Local-first software: You own your data, in spite of the cloud. Onward! 2019. <https://www.inkandswitch.com/essay/local-first/>

WHATWG. HTML Living Standard. The standard platform underlying browser-based documents and application interfaces. <https://html.spec.whatwg.org/>

Tauri. Architecture documentation. An example of a native application core paired with a system WebView frontend. <https://tauri.app/concept/architecture/>

Electron. Process model documentation. An example of installed desktop software using web rendering technology. <https://www.electronjs.org/docs/latest/tutorial/process-model>